

MediQueue — Medical Center Platform

Project Plan, Architecture & Implementation Roadmap

Table of Contents

- [1. Project Overview](#)
 - [2. Design Goals](#)
 - [3. Tech Stack](#)
 - [4. Project Structure](#)
 - [5. Database Schema](#)
 - [6. API Design](#)
 - [7. Core Business Logic](#)
 - [8. Real-time Events](#)
 - [9. Feature List](#)
 - [10. Implementation Phases](#)
 - [11. Infrastructure & DevOps](#)
 - [12. Testing Strategy](#)
 - [13. Future Roadmap](#)
-

1. Project Overview

MediQueue is a web-based platform for medical centers that manages appointment booking, walk-in patient queues, doctor location tracking, patient health history, and real-time clinic updates — built to serve both tech-savvy and non-tech-savvy patients equally.

The Problem It Solves

Problem	Solution
Overbooking and wasted slots	Fixed slots per hour with atomic Redis counters
Repeat no-shows	Automated penalty system with tiered restrictions
Patients who don't use apps	Walk-in queue managed by receptionist, SMS-only notifications
Doctors spending too much time on admin	Voice notes, templates, auto PDF prescriptions, AI summaries
Patients don't know how far/delayed a doctor is	Live doctor location map + real-time delay broadcasts
No central patient health history	Unified health records timeline with file attachments

2. Design Goals

For Patients

- Book appointments with instant slot availability
- See real-time doctor location on a map

- Subscribe to a doctor and get notified of delays
- View full health history and past prescriptions
- Receive reminders via SMS, WhatsApp, or in-app

For Non-Tech-Savvy Patients

- Walk directly into the clinic — receptionist adds them to the queue
- Get a physical queue number (printed or verbal)
- Optionally receive an SMS when their turn is near
- No app, no login, no internet required on their end

For Doctors

- Minimal time on admin work
- Speak visit notes instead of typing
- One-click templates for common conditions
- Auto-generated prescription PDFs
- AI-assisted visit summaries to review, not write
- Auto-scheduled appointment slots based on working hours

For Receptionists

- Manage walk-in queue from a single dashboard
- Call next patient with one button
- Clinic TV screen auto-updates with queue numbers

For Admins

- Configure slots per doctor per day
- View no-show statistics and penalty reports
- Reset patient penalties manually
- Manage doctors, clinics, and users

3. Tech Stack

Frontend

Layer	Technology	Reason
Framework	Next.js 14 (App Router)	SSR, file-based routing, built-in API routes for BFF
Language	TypeScript	End-to-end type safety with shared packages
Styling	Tailwind CSS	Utility-first, no runtime CSS overhead
Components	shadcn/ui	Accessible, composable, unstyled base
Server state	TanStack Query	Cache management, background refetch, optimistic updates
Forms	React Hook Form + Zod	Type-safe validation, minimal re-renders
Maps	Mapbox GL JS	Vector tiles, custom markers, routing queries
Real-time	Socket.IO client	Matches backend, auto-reconnect on mobile

Voice input	Web Speech API (browser-native)	Doctor dictation — free, no dependency
Video	LiveKit JS SDK	Open-source, in-browser video calls

Backend

Layer	Technology	Reason
Runtime	Node.js	Async-first, huge ecosystem
Framework	Fastify	2–3× faster than Express, built-in schema validation
Language	TypeScript	Shared types with frontend via monorepo
Auth	@fastify/jwt + @fastify/cookie	JWT access tokens + httpOnly refresh cookies
Validation	Zod	Shared schemas with frontend

Data Layer

Layer	Technology	Reason
Primary DB	PostgreSQL 16	JSONB for health records, row-level security, partitioning
Cache / Counters	Redis 7	Atomic slot counters (prevent overbooking), pub/sub, session store
Search	PostgreSQL FTS (→ Meilisearch later)	Full-text search on doctors and records
File storage	AWS S3 / Supabase Storage	Health report PDFs, lab result images

Real-time & Comms

Layer	Technology	Reason
WebSockets	Socket.IO	Rooms per doctor/clinic, auto-reconnect fallback to polling
Email	Resend	Appointment reminders, prescription delivery
SMS / WhatsApp	Twilio	Queue notifications and reminders for non-app users
Push notifications	Web Push API	Service worker, patient opt-in
Video	LiveKit	Open-source WebRTC, self-hostable
PDF generation	Puppeteer	Server-side prescription and report PDFs
AI summaries	Claude API (Anthropic)	AI-generated visit summaries and health insights

Infrastructure

Layer	Technology
Monorepo	Turborepo
Containers	Docker + Docker Compose
CI/CD	GitHub Actions
Cloud (MVP)	Railway / Render
Cloud (Scale)	AWS / GCP

4. Project Structure

medical-center/

├─ turbo.json

├─ package.json

├─ .env.example

├─ PROJECT_PLAN.md

├─ apps/

│ └─ web/

│ └─ app/

│ └─ (auth)/

│ └─ login/

│ └─ register/

│ └─ (patient)/

│ └─ dashboard/

│ └─ appointments/

│ └─ health-records/

│ └─ map/

│ └─ (doctor)/

│ └─ dashboard/

│ └─ schedule/

│ └─ patients/

│ └─ location/

│ └─ (receptionist)/

│ └─ queue/

│ └─ walk-ins/

│ └─ (admin)/

│ └─ dashboard/

│ └─ doctors/

│ └─ reports/

│ └─ queue-screens/

← monorepo root

← Next.js 14 frontend

← upcoming appointments, health summary

← browse slots, book, cancel, history

← health timeline, files, prescriptions

← live doctor map, subscribe, nearest

← today's schedule, quick stats

← full calendar, slot management

← patient list, health record entry

← set live location, broadcast delay

← walk-in queue board + call next

← add walk-in form

← stats, no-show rates, utilization

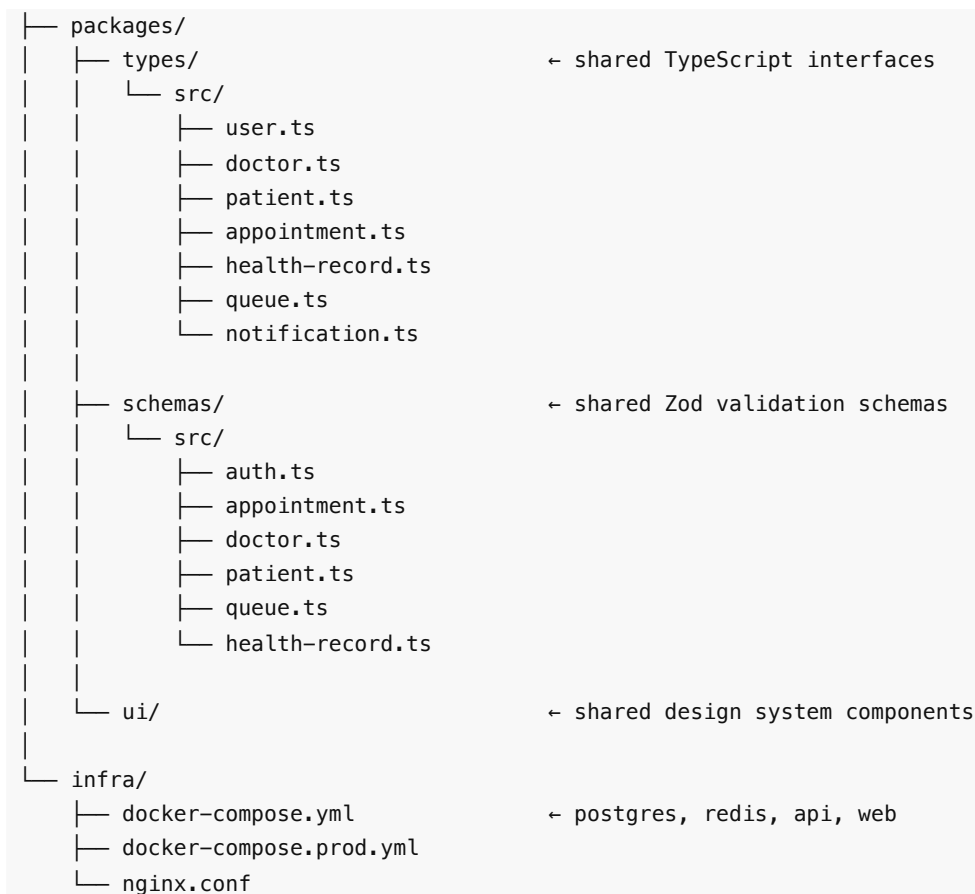
← manage doctors, slots config

← audit logs, exports

← manage TV display screens

	└─ display/		
	└─ [clinicId]/	← PUBLIC TV queue screen (no auth)	
	└─ components/		
	└─ maps/		
	└─ DoctorMap.tsx	← Mapbox GL JS map	
	└─ DoctorMarker.tsx	← colour-coded availability marker	
	└─ SubscribeButton.tsx	← subscribe to doctor updates	
	└─ appointments/		
	└─ SlotPicker.tsx	← date/hour slot grid	
	└─ PenaltyWarning.tsx	← warning banner for penalised patients	
	└─ BookingForm.tsx	← full booking flow	
	└─ health/		
	└─ HealthTimeline.tsx	← chronological record list	
	└─ RecordCard.tsx	← individual record display	
	└─ PrescriptionView.tsx		
	└─ queue/		
	└─ WalkInForm.tsx	← receptionist add-patient form	
	└─ QueueBoard.tsx	← live queue panel + call-next	
	└─ doctor/		
	└─ VoiceNoteInput.tsx	← Web Speech API dictation	
	└─ QuickTemplateSelector.tsx	← one-click note templates	
	└─ PrescriptionForm.tsx		
	└─ FollowUpButton.tsx	← trigger follow-up suggestion	
	└─ video/		
	└─ VideoRoom.tsx	← LiveKit video call	
	└─ WaitingRoom.tsx	← pre-call waiting UI	
	└─ lib/		
	└─ api-client.ts	← typed fetch wrapper	
	└─ socket.ts	← Socket.IO singleton client	
	└─ auth.ts	← login/register/refresh helpers	
	└─ speech.ts	← Web Speech API wrapper	
	└─ api/	← Fastify backend	
	└─ src/		
	└─ server.ts	← entry point, plugin registration, job start	
	└─ plugins/		
	└─ postgres.ts	← pg.Pool decorator	
	└─ redis.ts	← Redis client decorator	
	└─ auth.ts	← JWT plugin, authenticate & requireRole	
decorators	└─ cors.ts		
	└─ rate-limit.ts		
	└─ socket.ts	← Socket.IO server, room setup, WS auth	

		└─ modules/	
		└─ auth/	
		└─ auth.routes.ts	
		└─ auth.service.ts	← register, login, refresh token rotation
		└─ appointments/	
		└─ appointments.routes.ts	
		└─ appointments.service.ts	← book, cancel, no-show
		└─ slots.service.ts	← ATOMIC Redis slot reservation,
slot generation			
		└─ penalty.service.ts	← penalty scoring, late cancel
detection			
		└─ doctors/	
		└─ doctors.routes.ts	← list, profile, location, subscribe, delay
broadcast			
		└─ patients/	
		└─ patients.routes.ts	
		└─ health-records/	
		└─ records.routes.ts	← records, prescriptions, templates
		└─ queue/	
		└─ queue.routes.ts	
		└─ queue.service.ts	← add walk-in, call next, SMS trigger,
display broadcast			
		└─ video/	← LiveKit token generation
		└─ waitlist/	← slot waitlist management
		└─ ratings/	← post-appointment ratings
		└─ notifications/	← in-app notification CRUD
		└─ jobs/	
		└─ penalty.job.ts	← every 15min: detect no-shows, apply
penalties			
		└─ reminder.job.ts	← every 30min: 24hr + 2hr appointment
reminders			
		└─ slot-generator.job.ts	← weekly: auto-generate slots for all
doctors			
		└─ queue-clean.job.ts	← nightly: archive stale walk-in entries
		└─ shared/	
		└─ middleware/rbac.ts	
		└─ errors/	
		└─ migrations/	
		└─ 1_initial_schema.sql	← all 18 tables
		└─ tsconfig.json	



5. Database Schema

Entity Relationships

```
users —(1:1)— doctors
users —(1:1)— patients
doctors —(1:M)— doctor_slots
doctor_slots —(1:M)— appointments
patients —(1:M)— appointments
patients —(1:1)— penalty_profiles
patients —(1:M)— health_records
patients —(1:M)— prescriptions
doctors —(1:1)— doctor_locations
patients —(M:M)— doctors    [via doctor_subscriptions]
doctors —(M:M)— clinics    [via doctor_clinics]
appointments —(1:1)— video_rooms
appointments —(1:1)— ratings
walk_in_queue —(M:1)— doctors
walk_in_queue —(M:1)— clinics
```

Table Reference

Table	Key Columns	Purpose
-------	-------------	---------

users	id, email, password_hash, role, is_active	Base auth for all roles
doctors	user_id, specialization, slots_per_hour, working_hours JSONB	Doctor profile + slot config
clinics	name, address, latitude, longitude	Physical clinic locations
doctor_clinics	doctor_id, clinic_id	Doctor ↔ clinic many-to-many
patients	user_id, blood_type, allergies[], emergency_contact JSONB	Patient profile
penalty_profiles	patient_id, no_show_count, late_cancel_count, penalty_level (0–3), penalty_expires_at	No-show tracking
doctor_slots	doctor_id, slot_date, slot_hour, capacity, booked_count	Hour-level availability slots
appointments	patient_id, doctor_id, slot_id, status, is_late_number, slot_position	Bookings
waitlist	patient_id, slot_id, position, notified	Queue for fully-booked slots
health_records	patient_id, record_type, content JSONB, attachments[]	Visit notes, labs, vaccinations
prescriptions	patient_id, doctor_id, items JSONB, pdf_url	Structured prescriptions
appointment_templates	doctor_id, name, visit_note, prescription_items JSONB	Doctor quick-fill templates
doctor_locations	doctor_id, latitude, longitude, is_live	Live location broadcast
doctor_subscriptions	patient_id, doctor_id	Patient subscribes to doctor updates
walk_in_queue	patient_name, doctor_id, queue_number, status, sms_phone	Physical walk-in queue
video_rooms	appointment_id, room_name, started_at	LiveKit room tracking
ratings	patient_id, doctor_id, stars, comment	Post-appointment ratings
notifications	user_id, type, body, data JSONB, sent_via[]	In-app + push notifications
insurance_verifications	patient_id, appointment_id, status	Receptionist insurance checks
audit_log	user_id, action, resource, ip_address	HIPAA-readiness audit trail

Penalty Level Reference

Level	Trigger	Effect
0	Default	No restriction
1	no_show $\geq$ 2 OR late_cancel $\geq$ 3	Warning shown during booking
2	no_show $\geq$ 3 OR late_cancel $\geq$ 5	Auto-assigned last slot position (late number)
3	no_show $\geq$ 5	Booking suspended for 7 days

Penalty auto-expires after 60 days of good behaviour. Admin can reset at any time.

6. API Design

Auth

POST	/api/auth/register	Register patient / doctor / receptionist
POST	/api/auth/login	Returns access token + sets httpOnly refresh cookie
POST	/api/auth/refresh	Rotates refresh token
POST	/api/auth/logout	Clears refresh token from Redis + cookie
GET	/api/auth/me	Current user profile
POST	/api/auth/oauth/google	Google OAuth callback

Doctors

GET	/api/doctors	List (filters: specialty, available, lat/lng/radius)
GET	/api/doctors/:id	Doctor profile + rating + location
PUT	/api/doctors/me	Update own profile [DOCTOR]
PUT	/api/doctors/me/location	Update live location, broadcast to subscribers [DOCTOR]
POST	/api/doctors/me/delay	Broadcast delay notification to subscribers [DOCTOR]
POST	/api/doctors/:id/subscribe	Subscribe to doctor updates [PATIENT]
DELETE	/api/doctors/:id/subscribe	Unsubscribe [PATIENT]

Appointments & Slots

GET	/api/appointments	Patient's own appointments [PATIENT]
POST	/api/appointments	Book appointment (penalty-aware) [PATIENT]
PUT	/api/appointments/:id/cancel	Cancel (triggers late-cancel check)
PUT	/api/appointments/:id/complete	Mark complete [DOCTOR/RECEPTIONIST]
PUT	/api/appointments/:id/no-show	Mark no-show + apply penalty [DOCTOR]
GET	/api/appointments/schedule	Doctor's schedule by date [DOCTOR]
GET	/api/appointments/slots	Available slots for a doctor on a date
POST	/api/appointments/slots/generate	Generate slots for date range [ADMIN/DOCTOR]
GET	/api/appointments/penalty/me	Current patient's penalty profile [PATIENT]
PUT	/api/appointments/penalty/:id/reset	Admin reset [ADMIN]

Walk-in Queue

GET	/api/queue	Today's queue for a doctor [RECEPTIONIST/DOCTOR]
POST	/api/queue	Add walk-in patient (name + optional phone)
	[RECEPTIONIST]	
POST	/api/queue/call-next	Call next patient, SMS them, update display
	[RECEPTIONIST/DOCTOR]	
PUT	/api/queue/:id/status	Update queue entry status
GET	/api/queue/display/:clinicId	Public queue state for TV display (no auth)

Health Records

GET	/api/health-records/mine	Patient's own visible records [PATIENT]
GET	/api/health-records/patient/:id	Doctor views patient history [DOCTOR]
POST	/api/health-records	Create record post-appointment [DOCTOR]
POST	/api/health-records/prescriptions	Create prescription + queue PDF [DOCTOR]
POST	/api/health-records/templates	Save quick-fill template [DOCTOR]
GET	/api/health-records/templates	Doctor's saved templates [DOCTOR]

Patients

GET	/api/patients/me	Own profile [PATIENT]
PUT	/api/patients/me	Update profile [PATIENT]
GET	/api/patients/:id	Doctor/receptionist views patient [DOCTOR/RECEPTIONIST]

Notifications

GET	/api/notifications	List latest 50 [AUTH]
PUT	/api/notifications/:id/read	Mark read
PUT	/api/notifications/read-all	Mark all read

7. Core Business Logic

Slot Booking — Race Condition Prevention

1. Patient requests a slot
2. Check penalty level → level 3? reject
3. Redis Lua script (atomic):
GET slot:{id}:booked
IF count >= capacity → return "slot full" (409)
ELSE → INCR count, return new count
4. Async→sync new count to PostgreSQL booked_count
5. If penalty level 2 → set slot_position = capacity (late number)
6. INSERT appointment row
7. Broadcast slot_update event to doctor's Socket.IO room

Penalty Detection — Background Job (every 15 min)

1. `SELECT appointments WHERE status = 'scheduled'`
`AND appointment_date < NOW() - 30min`
2. Bulk `UPDATE → status = 'no_show'`
3. For each: `recordNoShow(patient_id)`
 - `no_show_count++`
 - `recompute penalty_level`
 - if `level = 3`: `set penalty_expires_at = NOW() + 7 days`

Walk-in Queue Flow

Receptionist:

1. Opens walk-in form
2. Enters patient name + optional phone number
3. Selects doctor
4. System assigns next queue number (`MAX + 1` for today)
5. Calculates estimated wait (`patients_ahead × 15 min avg`)
6. If phone provided → SMS: "Queue #7 at Dr. Smith's. Est. wait: 30 min."
7. Broadcasts `queue_update` to clinic's Socket.IO room
8. TV display screen updates automatically

Doctor / Receptionist calls next:

1. `POST /api/queue/call-next`
2. Mark current `in_progress` → `completed`
3. Mark next waiting → `called`
4. Broadcast updated queue state to TV display
5. If phone on file → SMS: "Please proceed to the consultation room"

Doctor Automation Workflows

Workflow	Trigger	Automation
Slot generation	Weekly cron job	Reads <code>working_hours</code> → generates slots for next 14 days
Visit note	Doctor speaks	Web Speech API transcribes to text field in real time
Template fill	Doctor clicks template	Pre-fills visit note + prescription items in one click
Prescription PDF	Doctor saves prescription	Puppeteer generates PDF, stores on S3, emails to patient
Follow-up suggestion	Doctor clicks "Follow-up in X weeks"	System queues notification to patient with pre-filtered slots
Visit summary	Appointment marked complete	Claude API generates a draft summary — doctor reviews it
Penalty detection	Background job every 15 min	No manual intervention required

SMS reminders	Background job every 30 min	Sent 24hr + 2hr before appointment automatically
---------------	-----------------------------	--

8. Real-time Events (Socket.IO)

Rooms

Room Key	Subscribers	Events
doctor:{doctorId}	Doctor (auto on login), subscribed patients	location, delay, status, slot_update
clinic_queue: {clinicId}	TV display screen, receptionist dashboard	queue_update
user:{userId}	Individual patient	appointment_reminder, penalty_issued, waitlist_available

Events Reference

Server → Client

Event	Payload	Trigger
doctor_location	{ doctorId, lat, lng, timestamp }	Doctor posts location update
doctor_delay	{ doctorId, delayMinutes, message }	Doctor broadcasts delay
doctor_status	{ doctorId, status }	Doctor connects / disconnects
slot_update	{ slotId, availableCount }	Slot booked or cancelled
queue_update	{ clinicId, doctorId, currentNumber, waitingCount }	Walk-in queue changes
appointment_reminder	{ appointmentId, timeUntil }	Background job fires
penalty_issued	{ penaltyLevel, expiresAt }	Penalty level increases
waitlist_available	{ slotId }	Cancellation opens a waitlisted slot

Client → Server

Event	Payload
subscribe_doctor	{ doctorId }
unsubscribe_doctor	{ doctorId }
subscribe_clinic_queue	{ clinicId }

9. Feature List

Core Features

- Fixed appointment slots per hour (configurable per doctor)
- Atomic slot reservation — no overbooking possible
- Tiered penalty system for repeated no-shows (4 levels)
- Physical walk-in queue — no patient account required
- Public TV queue display screen per clinic
- SMS notifications for walk-in patients (Twilio)
- Doctor live location map (Mapbox)
- Subscribe to nearest doctor + delay notifications
- Patient health records timeline
- Appointment booking history

Doctor Automation

- Voice-to-text visit notes (Web Speech API)
- Quick-fill templates for common conditions
- One-click follow-up scheduling trigger
- AI-generated visit summary (Claude API, doctor reviews)
- Auto prescription PDF generation + email to patient
- Auto weekly slot generation from working hours
- Batch appointment status updates

Additional Features

- Telemedicine / video consultation (LiveKit)
- Structured prescription management (printable PDFs)
- Waitlist system for fully-booked slots
- Post-appointment patient ratings (1–5 stars)
- Multi-clinic / branch support (doctors ↔ clinics)
- Insurance verification workflow (receptionist marks status)
- WhatsApp notifications via Twilio (for non-app patients)
- In-app notification centre
- Web Push notifications (service worker)
- Audit log for HIPAA readiness
- Google OAuth for patient sign-up

10. Implementation Phases

Phase 0 — Foundation

Duration: Week 1–2

Goal: Runnable skeleton with working auth

Tasks:

- ☐ Initialize Turborepo monorepo
- ☐ Docker Compose: PostgreSQL 16, Redis 7, Fastify API, Next.js web
- ☐ Run `migrations/1_initial_schema.sql` on first start
- ☐ Implement `users` table + JWT auth (register, login, refresh, logout)

- ☐ Role middleware (RBAC guard: admin, doctor, patient, receptionist)
- ☐ Shared `packages/types` with all base entity types
- ☐ Shared `packages/schemas` with Zod validation
- ☐ Next.js layout + auth context + role-based protected routes
- ☐ CI pipeline: ESLint, TypeScript checks

Deliverable: Login/register working, roles enforced, `/healthz` returns 200

Phase 1 — Appointment Booking Core

Duration: Week 3–5

Goal: End-to-end appointment booking with race-condition protection

Tasks:

- ☐ Slot generation from `working_hours × slots_per_hour` (weekly cron)
- ☐ Redis atomic slot reservation (Lua script)
- ☐ Slot availability API + `SlotPicker` UI component
- ☐ `BookingForm` with doctor and date selection
- ☐ Doctor schedule view with quick-action buttons (confirm / complete / no-show)
- ☐ Waitlist system (Redis sorted set, notify on cancellation)
- ☐ Receptionist walk-in queue entry form

Deliverable: Patient books → slot count decrements → no overbooking under concurrent load

Phase 2 — Penalty System

Duration: Week 5–6

Goal: Automated no-show management

Tasks:

- ☐ `penalty.job.ts` background job (every 15 min)
- ☐ No-show detection + penalty level recalculation
- ☐ Late cancellation detection (< 2hr before)
- ☐ Penalty-aware booking: late number assignment
- ☐ `PenaltyWarning` UI component for patients
- ☐ Admin penalty reset endpoint + UI

Deliverable: Patient with 3 no-shows gets late number on next booking; level 3 blocks booking

Phase 3 — Physical Walk-in Queue

Duration: Week 6–8

Goal: Serve patients who come directly to the clinic with no technology

Tasks:

- ☐ Walk-in queue API (`addWalkIn` , `callNextPatient` , `getTodayQueue`)
- ☐ `WalkInForm` — receptionist adds patient (name + optional phone only)
- ☐ `QueueBoard` — receptionist dashboard with "Call Next" button

- ☐ Public TV display screen `/display/[clinicId]` — no auth, Socket.IO subscriber
- ☐ Queue number auto-increment per doctor per day
- ☐ Estimated wait calculation (avg consultation time × queue position)
- ☐ SMS notification on check-in and on call via Twilio
- ☐ `queue-clean.job.ts` — nightly archive of stale entries

Deliverable: Receptionist adds walk-in → TV screen updates live → patient gets SMS when called

Phase 4 — Real-time Doctor Location & Map

Duration: Week 8–10

Goal: Live doctor map with real-time subscriptions

Tasks:

- ☐ Mapbox GL JS integration (`DoctorMap` , `DoctorMarker` components)
- ☐ `doctor_locations` table + GET/PUT endpoints
- ☐ Doctor "Go Live" page — browser Geolocation API → POST every 30s
- ☐ Nearest doctor query (PostGIS `ST_DWithin` or Haversine formula)
- ☐ Subscribe/unsubscribe buttons on map markers
- ☐ Doctor delay broadcast → `doctor_delay` Socket.IO event to room
- ☐ Delay notification creates rows in `notifications` table

Deliverable: Patient opens map → sees live doctors → subscribes → gets delay notification in real time

Phase 5 — Health Records & Doctor Automation

Duration: Week 10–13

Goal: Complete patient history + minimal-effort doctor workflows

Tasks:

- ☐ Health records CRUD API
- ☐ `VoiceNoteInput` — Web Speech API dictation in visit note field
- ☐ `QuickTemplateSelector` — one-click pre-fill from saved templates
- ☐ Template save/delete UI
- ☐ `FollowUpButton` — doctor sets follow-up weeks → patient gets slot suggestions
- ☐ AI summary job — Claude API generates draft visit summary post-appointment
- ☐ File upload: presigned S3 URLs for lab result PDFs / images
- ☐ Patient `HealthTimeline` UI — chronological records view
- ☐ Prescription module: structured entry → auto Puppeteer PDF → S3 → email

Deliverable: Doctor speaks note, selects template, saves — AI draft ready; patient sees timeline

Phase 6 — Telemedicine / Video

Duration: Week 13–15

Goal: In-browser video consultations

Tasks:

- ☐ LiveKit server setup + token generation API
- ☐ "Virtual appointment" flag on booking form
- ☐ `WaitingRoom` UI — patient waits, doctor joins to start call
- ☐ `VideoRoom` component — LiveKit JS SDK
- ☐ Post-call: auto-save visit note prompt for doctor
- ☐ Video room status in `video_rooms` table

Deliverable: Patient books virtual appointment → joins waiting room → video call starts on doctor join

Phase 7 — Multi-clinic, Ratings & Insurance

Duration: Week 15–17

Goal: Support multiple branches, patient feedback, insurance checks

Tasks:

- ☐ `clinics` table + doctor ↔ clinic M:M
- ☐ Map shows clinic locations (not just individual doctor positions)
- ☐ Post-appointment rating (unlocked 30 min after `completed`)
- ☐ Doctor profile: aggregate star rating display
- ☐ Admin: flag/remove abusive reviews
- ☐ Insurance verification workflow — receptionist marks pending/verified/rejected

Deliverable: Multi-clinic map view, patients can rate doctors, receptionists verify insurance

Phase 8 — Notifications, Reminders & WhatsApp

Duration: Week 17–18

Goal: Multi-channel proactive communication

Tasks:

- ☐ `reminder.job.ts` — 24hr + 2hr before via Resend (email) + Twilio (SMS)
- ☐ WhatsApp Business API via Twilio — queue notifications + appointment reminders
- ☐ In-app notification bell (Socket.IO `user:{id}` private room)
- ☐ Web Push service worker — opt-in push for patients
- ☐ Notification preferences page (patient controls channels)

Deliverable: Patient gets reminder SMS + email + WhatsApp + in-app notification before appointment

Phase 9 — Admin Dashboard & Production Hardening

Duration: Week 18–20

Goal: Admin control panel + production-ready security and performance

Tasks:

- ☐ Admin dashboard: daily bookings, no-show rate chart, slot utilisation heatmap
- ☐ Slot configuration UI (admin sets `slots_per_hour` per doctor per day)
- ☐ Audit log viewer (HIPAA readiness)
- ☐ Rate limiting review (all endpoints)

- ☐ Input sanitisation audit (SQL injection, XSS)
- ☐ Load testing with k6 (concurrent booking race conditions)
- ☐ Accessibility audit (WCAG 2.1 AA)
- ☐ Database index audit + slow query profiling
- ☐ Staging environment with anonymised seed data

Deliverable: Platform is production-deployable, tested under load, accessible

11. Infrastructure & DevOps

Local Development

```
# Clone and install
npm install

# Copy env file and fill in secrets
cp .env.example .env

# Start all services
docker compose -f infra/docker-compose.yml up

# Database schema auto-applies on first postgres start
# API: http://localhost:4000
# Web: http://localhost:3000
# TV Display: http://localhost:3000/display/{clinicId}
```

Environment Variables

Variable	Used by	Required
DATABASE_URL	API	Yes
REDIS_URL	API	Yes
JWT_SECRET	API	Yes
COOKIE_SECRET	API	Yes
TWILIO_ACCOUNT_SID	API	Phase 3+
TWILIO_AUTH_TOKEN	API	Phase 3+
TWILIO_PHONE_NUMBER	API	Phase 3+
TWILIO_WHATSAPP_FROM	API	Phase 8
RESEND_API_KEY	API	Phase 8
NEXT_PUBLIC_MAPBOX_TOKEN	Web	Phase 4
LIVEKIT_API_KEY	API	Phase 6
LIVEKIT_API_SECRET	API	Phase 6

ANTHROPIC_API_KEY	API	Phase 5
STORAGE_BUCKET	API	Phase 5

Deployment (MVP)

1. Push to GitHub
 2. GitHub Actions runs: lint → type-check → tests
 3. On merge to `main` : deploy API to Railway, web to Vercel
 4. PostgreSQL and Redis as managed services (Railway or Supabase)
-

12. Testing Strategy

Unit Tests (Vitest)

- `penalty.service.ts` — all threshold transitions (0→1→2→3, auto-expiry)
- `slots.service.ts` — slot capacity math, late number assignment
- `queue.service.ts` — queue number sequencing, estimated wait calculation
- Zod schemas — malformed inputs rejected correctly

Integration Tests (Vitest + Supertest)

- 10 concurrent booking requests for a 1-capacity slot → exactly 1 succeeds
- Penalty accumulation: simulate 3 no-shows → verify level 2 is set
- Walk-in added → Socket.IO `queue_update` emitted to clinic room
- Refresh token rotation: old token rejected after use

E2E Tests (Playwright)

- Full booking flow: register → book → doctor marks complete → health record visible
- Walk-in flow: receptionist adds patient → TV display updates → call next → SMS sent
- Penalty flow: 3 no-shows → next booking gets late number
- Map flow: patient opens map → subscribes to doctor → doctor broadcasts delay → patient receives event

Load Tests (k6)

- 100 concurrent bookings for 1-capacity slot → assert exactly 1 success
- 500 patients in one Socket.IO doctor room → broadcast latency < 500ms
- 50 concurrent walk-in additions → queue numbers are sequential, no duplicates

Manual QA Checklist

- ☐ Role isolation: patient cannot access `/admin/*` or another patient's records
 - ☐ Walk-in patient receives SMS within 30 seconds of being called
 - ☐ TV display updates within 1 second of queue change
 - ☐ Map loads and shows doctors on a simulated 3G connection
 - ☐ Doctor location updates reflect on map within 5 seconds
 - ☐ Prescription PDF generates and emails within 60 seconds
 - ☐ Voice note transcription works in Chrome and Safari
 - ☐ Penalty reset by admin clears warning UI immediately
-

13. Future Roadmap

Post-Launch (3–6 months)

Feature	Description
Mobile App	React Native — reuses packages/types and packages/schemas
AI Symptom Checker	Patient describes symptoms before booking → AI suggests specialization
Lab Integration	HL7 FHIR API for ingesting lab results directly into health records
Billing Module	Per-consultation invoicing, payment integration (Stripe)
Staff Roster	Doctor leave management, receptionist shift scheduling

Scale Considerations

Concern	Solution
High booking concurrency	Redis Lua script already in place; add more Redis replicas
Large appointments table	Partition by appointment_date (monthly)
Many Socket.IO connections	Add Redis adapter for Socket.IO; scale API horizontally
File storage	CDN in front of S3 for health record attachments
AI summary cost	Batch summaries overnight instead of real-time